
Developing GPU Kernels with Agentic Loops

HAN Lab Kernel Mafia

Abstract

Generating high-performance CUDA kernels is not a one-shot code completion task. It is an iterative engineering process that requires planning edits, modifying code, measuring performance, inspecting profiling evidence, and refining the implementation. Current LLM-based systems often hide this loop behind rigid API pipelines, which limits the agent’s access to the development environment, domain knowledge, and fine-grained feedback. We adapt KDA: Kernel Design Agents [1], an agent workflow that brings this optimization loop into a real development environment. At the core of Kernel Design Agents is the Humanize coder-verifier loop [31], where a coding agent proposes and implements kernel changes while an independent verifier checks correctness and performance. To make the agent more effective on GPU kernels, we further equip it with two domain skills: KernelWiki, a retrievable knowledge base built from production kernels, public submissions, baselines, and documentation; and `ncu-report-skill`, which converts Nsight Compute reports into actionable performance feedback. In the Pure Agent track of the MLSys-2026 NVIDIA Blackwell Kernel Competition, Kernel Design Agents outperformed the provided FlashInfer baselines on three submitted kernels, achieving mean speedups of $19.08\times$ on DSA Indexer, $4.54\times$ on DSA Attention, and $1.92\times$ on GDN Prefill. We have released the code and prompts at <https://github.com/mit-han-lab/mlsys2026-flashinfer-constest>.

1 Introduction

GPU kernels such as GEMM, convolution, and attention are the computational foundation of modern AI systems. Yet writing fast kernels is still labor-intensive. It requires strong architecture-specific expertise, and the burden grows with each new generation of NVIDIA hardware. For example, moving from Hopper (SM90), which introduced Tensor Memory Accelerator (TMA) and warp specialization, to Blackwell (SM100) requires engineers to master new features such as `tcgen05`, Tensor Memory, and NVFP4. Tensor compilers such as TVM [8], Triton [33], and TorchInductor [2] automate parts of kernel development, but they still rely heavily on expert-written templates and often fall short of hand-optimized production kernels. Recent LLM-based search systems push beyond static compiler schedules: K-Search uses a co-evolving world model to separate high-level optimization planning from low-level program instantiation, while AVO turns autonomous coding agents into evolutionary variation operators that propose, repair, critique, and verify kernel edits [7, 9]. These systems show that agentic search can discover nontrivial kernel optimizations, yet they still center on specialized search loops around selected kernels and benchmark configurations rather than placing the agent in a full production-style development environment with repository state, community knowledge, profiler evidence, and workload-wide adaptation. This remaining gap makes end-to-end kernel generation with Large Language Models (LLMs) an attractive direction.

Recent LLM-based kernel generation systems have made encouraging progress, including training-based methods [6, 12, 38] and training-free methods [7, 11, 18, 23]. Despite these differences, most systems still follow a legacy *API-calling* paradigm. Meanwhile, the broader software-engineering community has moved toward what is often called *vibe coding* or agentic coding: the developer

Developing GPU Kernels with Agentic Loops

Humanize Kernel Agent Loop lets an AI agent build fast GPU kernels like a human engineer.

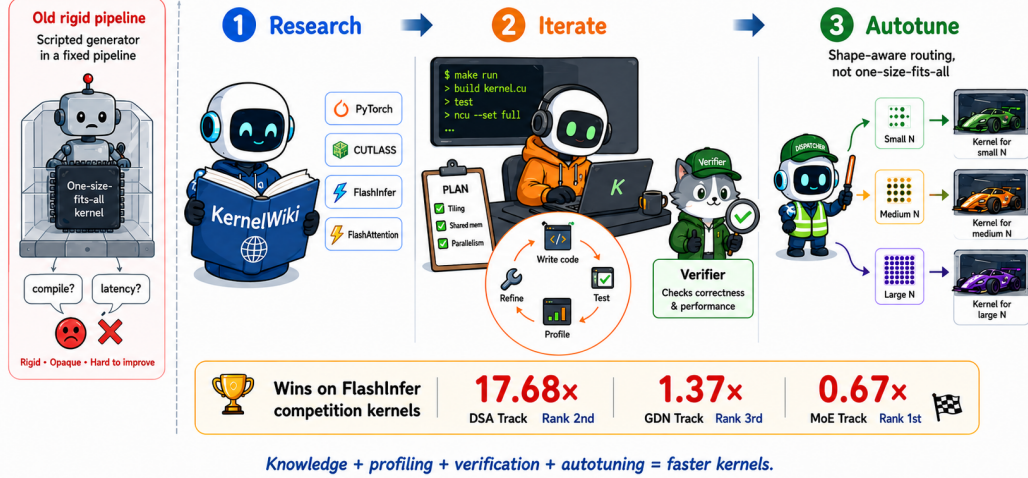


Figure 1: The overview of our kernel development agentic loop. The agent works inside a real development environment, where it plans kernel changes, retrieves community knowledge from KernelWiki, edits and verifies CUDA code, launches fine-grained Nsight Compute analysis with `ncu-report-skill`, and iterates until correctness and performance goals are met.

describes the desired outcome in natural language, while an agentic environment reads the codebase, plans changes, invokes tools, edits files, runs tests, and iterates under human supervision [3, 4, 24, 26]. In this view, an effective coding agent is not only a stronger model, but a composition of model, tools, and workflow. Existing kernel-generation pipelines have not fully absorbed this shift. The model mainly acts as a code generator, while a fixed human-written pipeline controls tool calls, file edits, profiling, validation, rollback, and search. This design hides the most important part of CUDA kernel engineering: the iterative loop of planning, implementation, measurement, diagnosis, and revision. Fast kernels rarely appear from one clever prompt. They emerge from this loop.

The main limitation is not model quality alone. It is also the interface between the model and this engineering loop. First, systems based on isolated API calls depend heavily on the frozen knowledge of the model. This makes it hard to use fast-changing architecture techniques, profiling heuristics, and implementation patterns. Second, tool usage is predefined rather than chosen by the agent. The process becomes rigid: the model cannot freely decide what to inspect, which file to modify, when to profile, or how to branch and backtrack from intermediate evidence. Third, these systems usually optimize against weak feedback, such as compilation success or a few scalar performance numbers. Human experts use richer signals from profilers, code context, and iterative debugging inside a real repository. Without an agentic loop, the model works with much less information than an experienced kernel engineer. Finally, many pipelines evaluate kernels on a small set of fixed shapes to keep iteration fast. This can produce solutions that perform well on a narrow benchmark subset but fail across the target workload distribution in deployment.

Our central observation is that the agentic loop is the core primitive for CUDA kernel generation. Frontier models such as Claude Opus 4.6[5] and GPT-5.4[25] are no longer limited to one-shot code completion. When placed in a real development environment with access to repository state, community kernel knowledge, profiler outputs, and domain-specific tools, they can plan, implement, inspect, debug, and refine kernels in a more human-like way. In this setting, the loop is not glue code around the model. It is the mechanism that turns model knowledge into working, fast CUDA code.

To this end, we adapt Kernel Design Agents, an agentic-loop-centric kernel generation framework built around three techniques. First, we use the Humanize agentic loop [31], which lets agents lead development through planning, execution, verification, and review. This loop matches kernel

engineering closely: the agent forms a concrete plan, edits CUDA code, runs correctness and performance checks, inspects failures, and revises the plan with new evidence. Second, we combine uses KernelWiki to absorb kernel knowledge from production pull requests, public submissions, baselines, and documentation in popular repositories such as SGLang, vLLM, TensorRT-LLM, PyTorch, and more. This gives the agent grounded examples beyond its parametric memory. Third, we utilize `ncu-report-skill` to launch fine-grained Nsight Compute analysis. The skill extracts metrics, stall hotspots, PM-sampling timelines, and source-level evidence, helping the agent locate and reduce performance bubbles as much as possible. Together, these techniques make kernel generation an agent-driven engineering loop rather than an API pipeline that still depends on frequent human steering.

Our main contributions are summarized as follows:

- We identify the hardcoded API-calling paradigm as the root cause of process, knowledge, feedback, and workload limitations in existing LLM-based kernel generation systems. We argue that the agentic loop is the central mechanism for CUDA kernel generation.
- We adapt KDA: Kernel Design Agents [1], an agentic-loop-centric framework built on the Humanize agentic loop [31]. The agent can plan, edit, verify, profile, and debug kernels inside a real repository instead of relying on API calls and frequent human steering.
- We introduce KernelWiki, which absorbs production-grade kernel knowledge from popular repositories such as SGLang, vLLM, TensorRT-LLM, PyTorch, and more, and makes it retrievable by the agent during development.
- We introduce `ncu-report-skill`, which turns fine-grained Nsight Compute evidence into concrete optimization guidance for reducing stalls, bubbles, and other runtime bottlenecks.
- In the MLSys-2026 NVIDIA Blackwell Kernel Competition, our submission ranked first in the MoE Track, second in the DSA Track, and third in the GDN Track under the Full-Agent Approach. It also outperformed the provided FlashInfer baselines on three kernels, with mean speedups of $19.08\times$ on DSA Indexer, $4.54\times$ on DSA Attention, and $1.92\times$ on GDN Prefill. After improving the workflow, we can further improve kernels than the submitted competition version.

2 Related Work

LLMs for GPU Kernel Generation. KernelBench [27] established GPU kernel generation as a concrete target for code-capable LLMs and showed that even frontier models remain far from reliably producing efficient kernels. Follow-up benchmarks and surveys broadened this picture from CUDA to Triton operators and more general automated kernel engineering settings [20, 35, 40]. Subsequent generation systems have pursued two broad directions. Training-based methods adapt models to kernel programming through supervised fine-tuning, reinforcement learning, or trajectory distillation [6, 12, 21, 38]. Training-free methods instead combine a frozen model with search or refinement procedures [7, 10, 17, 22, 23]. Our work belongs to the training-free line, but differs in that we do not adapt model weights and do not wrap the model in a fixed search scaffold; instead, we let a frontier coding agent operate directly inside a real development environment.

Agentic Kernel Optimization, Search, and Verification. Recent work increasingly frames kernel engineering as an agentic systems task rather than a one-shot code-generation problem. Representative directions include multi-agent and memory-based frameworks [11, 30, 32, 37], evolutionary or verifier-guided search [7, 12, 18, 23, 29], and profiling- or hardware-feedback-guided refinement [14, 15, 19, 34, 41]. These systems substantially advance the state of the art, but most still expose the model through a human-authored pipeline in which tool usage, search order, and rollback logic are prescribed in advance. In contrast, our framework centers on an agent that can decide when to inspect files, profile, edit, backtrack, and branch, while augmenting that autonomy with explicit kernel knowledge retrieval and expert-distilled profiling interpretation.

GPU DSLs, Compilers, and Autotuning. Our work also relates to the broader literature on automating GPU performance engineering through domain-specific abstractions and schedule search. Systems such as TVM [8], Triton [33], TileLang [36], and TorchInductor [2] reduce the burden of kernel development, but strong performance still depends on expert templates, schedule design,

and architecture-aware tuning. The data and model ecosystem around this space has also expanded, including kernel corpora and specialized kernel-oriented LLMs [13, 28]. Autotuning therefore remains central, both in compilers and in LLM-based systems such as AutoTriton [21]. Our agentic autotuning shares this goal, but targets a different regime: rather than tuning a fixed implementation on a few benchmark instances, the agent scans the target shape distribution and constructs shape-aware routers over specialized kernels or configurations. This emphasis is particularly important for deployment-oriented settings such as FlashInfer-Bench [39], where generated kernels must remain robust across realistic serving traces.

3 Method

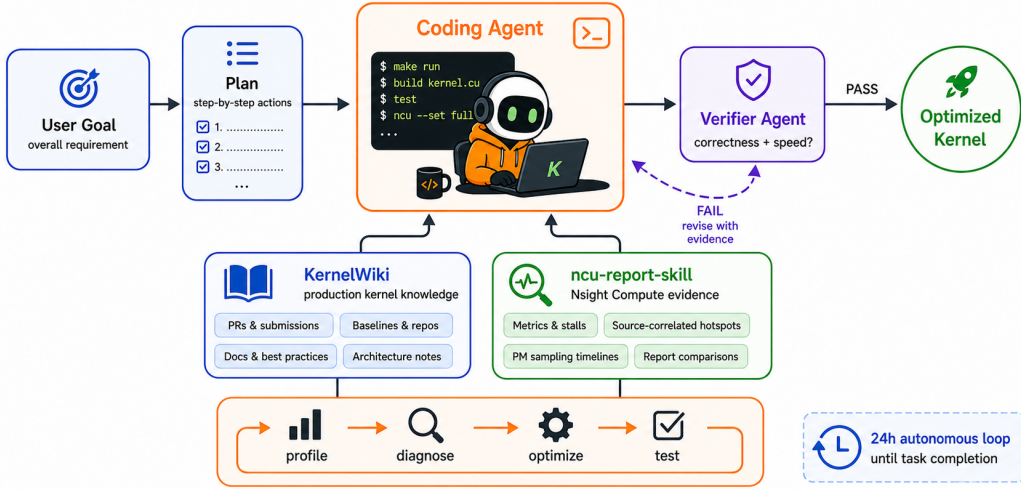


Figure 2: Humanize Kernel Agent Loop. Kernel Design Agents adapts the Humanize plan–execute–verify loop [31] to autonomous GPU kernel development. The workflow turns the user goal into a gated plan with explicit acceptance criteria, lets a writer agent edit code, run tools, and profile kernels inside the target development environment, and then requires an independent acceptance agent to verify each claimed step before progress is accepted. This independent verifier is critical for long-running kernel work: by catching incomplete validation, incorrect baselines, and unsupported performance claims, it allows the agent loop to run for 24 hours without routine human intervention. KernelWiki and ncu-report-skill are available throughout the loop as domain-specific skills.

We use Kernel Design Agents, a training-free framework for end-to-end GPU kernel generation and optimization in the target development environment. Rather than treating the model as a completion API inside a fixed outer pipeline. The system is designed to close the four limitations discussed in Section 1: process rigidity, stale architecture knowledge, impoverished feedback, and weak workload coverage.

At a high level, our solution has three stages shown in Figure 3: research, iterative refinement, and autotuning. The agent first retrieves architecture-specific knowledge from KernelWiki and inspects the target repository. It then iteratively writes, tests, and profiles candidate kernels inside the repository. Finally, it scans the target shape distribution and synthesizes a shape-aware routing policy instead of committing to a single kernel configuration.

3.1 Agentic Plan–Execute–Verify Loop

The core execution substrate of our solution is a Humanize-style [31] plan–execute–verify loop. We follow this mechanism for two practical reasons. First, long-running agents often stop unexpectedly because they encounter permission checks, tool failures, or ambiguous repository states that would normally trigger a request for human confirmation. Our goal, however, is to minimize human

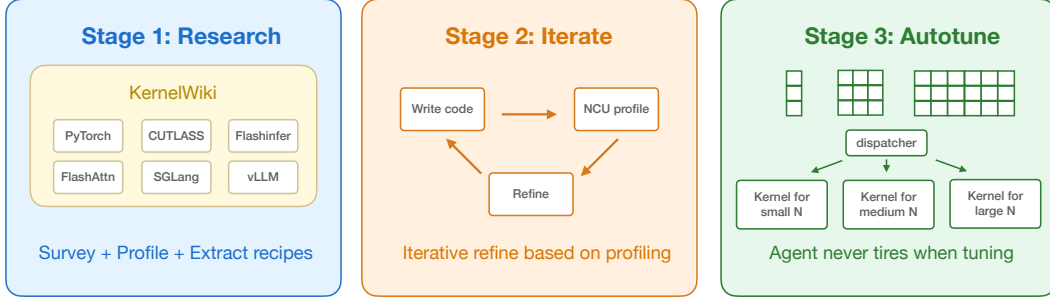


Figure 3: The three-stage pipeline of our solution. Stage 1 retrieves production practices and case studies from KernelWiki. Stage 2 performs iterative implementation and profiling-guided refinement in the target repository. Stage 3 performs agentic autotuning over the target shape distribution and emits a shape-aware dispatcher instead of a single monolithic kernel.

intervention. We therefore let Claude first consult Codex when it gets stuck; only unresolved issues are escalated to the human user. This keeps the agent loop running much longer without supervision.

Second, built-in planning modes are often unreliable when used directly for long-horizon kernel optimization. We repeatedly observed cases where the writer agent believed it had completed a task even though key requirements were still unmet, or where it optimized against the wrong acceptance target altogether. In the worst case, the agent effectively hacks its own plan by declaring success after partial validation. For example, it may check numerical correctness on only a small subset of workloads, or report speedups against an incorrect baseline. These failures are not rare corner cases; they are a recurring systems issue in autonomous kernel work.

Algorithm 1 Humanize Loop for Kernel Generation

Require: Kernel definition K , PyTorch reference R , workload distribution W

Ensure: Kernel implementation $\hat{K}$ that passes correctness checks on W and minimizes latency

```

1: // Planning
2: Human, Claude, and Codex co-design a plan  $\mathcal{P} = \{(t_i, ac_i)\}_{i=1}^N$ 
3: // The plan may describe any search strategy in natural language
4: // Execution
5: for each task-acceptance pair  $(t_i, ac_i) \in \mathcal{P}$  do
6:   done  $\leftarrow$  false
7:   while not done do
8:     Claude executes  $t_i$  in the repository
9:     // inspect/edit code, compile, test, benchmark, profile, query KernelWiki
10:    verdict, feedback  $\leftarrow$  Codex reviews evidence against  $ac_i$ 
11:    if verdict = pass then
12:      done  $\leftarrow$  true
13:    else
14:      Claude refines the kernel using feedback
15:      // if blocked, Claude consults Codex; ask human only if still unresolved
16:    end if
17:  end while
18: end for
19: return final kernel, dispatcher, and tuning decisions

```

To make this work in practice, KDA decomposes each task into small verifiable steps with explicit acceptance criteria. This turns the plan into an executable contract rather than a loose checklist.

Execution then proceeds inside the repository using a writer agent with access to ordinary development tools. Tool use is not hardcoded into a fixed sequence: the agent can inspect files, edit code, compile kernels, run tests, profile bottlenecks, and backtrack when needed. Every major step is gated by an independent verifier agent, which checks the claimed progress against code diffs, test results, profiler evidence, and repository state. In practice, this catches incomplete correctness validation, incorrect

benchmark baselines, missing integration, and unsupported performance claims, and serves as our main defense against reward hacking.

3.2 KernelWiki: Retrieval of Production-Grade Kernel Knowledge

An agent is only as strong as the knowledge it can bring into the loop. Our first attempt was the obvious one: expose the agent to official documentation and a few example kernels. In practice this was not enough. Official docs teach APIs and hardware primitives, but much of the knowledge behind fast production kernels is not written there.

To address this gap, we built KernelWiki, inspired by Karpathy’s LLMWiki idea [16] but specialized for GPU kernels. Instead of centering the knowledge base on tutorials and papers alone, we collect evidence from real production environments. Concretely, we gathered Hopper- and Blackwell-related pull requests from major repositories over the last two years, including PyTorch, CUTLASS, SGLang, vLLM, FlashInfer, and DeepGEMM. We also collected competition problems, baselines, and public submissions from GPU Mode and FlashInfer MLSys 2026.

KernelWiki organizes this material into a structured knowledge base for agents. It records both human-readable summaries and machine-readable metadata, allowing the agent to retrieve grounded examples instead of relying on parametric memory alone. The system also preserves traceability to the underlying PRs, docs, blogs, and contest artifacts.

In practice, this knowledge base often provides the key step that turns a stalled search into a productive one. For example, when implementing the DeepSeek sparse attention TopK indexer, the agent retrieved important ideas from the vLLM codebase and used them to guide subsequent optimization; see section 4.3.1.

3.3 Profiling-Grounded Iteration with `ncu-report-skill`

While KernelWiki supplies architecture knowledge, it does not explain why a specific candidate kernel is slow in a specific repository. For that, KDA relies on `ncu-report-skill`, a profiling skill specialized for Nsight Compute on B200 (`sm_100`). This skill was distilled from the workflow of a professional GPU kernel engineer: we asked what signals they inspect in practice, what failure modes those signals distinguish, and how they turn profiler evidence into concrete optimization decisions. The guiding principle is the same as our use of an agentic development environment: whatever information a strong human expert would want to see, the agent should also be able to access. As a result, the skill follows a simple rule, namely profile first, diagnose second, optimize third.

`ncu-report-skill` therefore standardizes profiling into a reproducible workflow that builds a harness, collects Nsight Compute reports, and extracts evidence such as key metrics, source-correlated stall hotspots, PM-sampling timelines, and report comparisons. Compared with pipelines that provide only coarse global statistics, this gives the agent a much richer view of what the kernel is doing at runtime, especially through PM sampling and inline-PTX-level hotspot analysis. The output is not merely “memory-bound” or “compute-bound”; it is a concrete inference chain from measured counters to likely mechanism to actionable fix. This is especially important for phenomena such as tail effects, uneven block lifetimes, or inline-PTX hotspots that are hard to diagnose from aggregate summaries alone.

3.4 Agentic Autotuning and Shape-Aware Routing

After the agent has produced a correct and reasonably optimized kernel, Kernel Design Agents enters an autotuning phase. Traditional pipelines often evaluate only a small fixed set of benchmark shapes, which encourages overfitting. Our setting is different: the target is a distribution of shapes derived from realistic workloads, and good solutions may require different kernel configurations or even different implementations in different regions of that distribution.

We therefore treat kernel selection itself as an agentic problem. The agent samples representative shapes, benchmarks candidate configurations, and incrementally builds a performance map over the target distribution. The important point is that the agent is not restricted to selecting parameters of one canonical kernel; it may decide that different shape regimes should use structurally different implementations. The final output is thus a *shape-aware router* that dispatches each input shape

to the best specialized kernel or configuration. This directly addresses the workload-generalization limitation identified in Section 1; see sections 4.3.1 and 4.3.3 for two examples where this routing strategy is important.

4 Experiments

4.1 Experimental Setup

Hardware and Software. All experiments were conducted on two bare-metal NVIDIA B200 GPUs (sm_100a). We used CUDA 13.2 and otherwise aligned our environment with the official FlashInfer evaluation setup.¹ Specifically, we matched locked B200 clocks, Python 3.12, PyTorch 2.12.0+cu132, and Triton 3.6.0, with FlashInfer and FlashInfer-Bench built from source. The environment also includes `cupti-python`, `deep-gemm`, `helion`, TVM, and CuTe DSL support.

Evaluation Protocol. We evaluate only on the workload set used in the FlashInfer competition, always against the corresponding official FlashInfer baseline. The benchmark reports latency only, so our main metric is average latency over the official workloads together with the derived speedup over the official baseline. We separate two settings. The first is the submitted competition setting, which reports the version of Kernel Design Agents submitted under the original competition timeline. The second is a post-competition controlled setting, where we rerun Kernel Design Agents and K-Search [7] under the same strict 48-hour budget to study the method itself and isolate the contribution of individual skills.

Agent Resources. The agent loop used two Claude Max subscription accounts and one Codex Pro subscription account, plus \$1,500 of Claude API credits to keep long-running experiments alive when the subscription accounts hit usage limits.

4.2 Competition Results

We first tested Kernel Design Agents in the MLSys-2026 NVIDIA Blackwell Kernel Competition. In the full-agent setting, Kernel Design Agents ranked #1 in the MoE track, #2 in the DSA track, and #3 in the GDN track. These track rankings compare against other competition submissions, while Table 1 reports the per-kernel speedup of the submitted competition version of Kernel Design Agents over the corresponding official FlashInfer baseline. Under the original competition timeline, the submitted system outperformed the official baseline on three kernels, with the largest gains on DSA Indexer and DSA Attention. These results show that the agentic workflow can produce high-performance kernels under competition pressure.

Table 1: Submitted competition results. Arithmetic-mean speedup of the submitted Kernel Design Agents system over the corresponding official FlashInfer baseline on the five competition kernels.

Kernel	Mean Speedup vs. Baseline
DSA Attention	4.5425
DSA Indexer	19.0758
MoE FP8	0.6504
GDN Decode	0.8023
GDN Prefill	1.9158

4.3 Competition Kernel Case Studies

The competition contains three tracks and five kernels. Below we summarize each operator and its main optimization trajectory.

¹<https://github.com/flashinfer-ai/flashinfer-bench-starter-kit/blob/main/EVALUATION.md>

4.3.1 DSA Track

DSA TopK Indexer. The DSA TopK indexer computes sparse scores over the KV cache and returns the top- K token indices:

$$s_j = \sum_h w_h \cdot \text{ReLU}(q_h^\top K_{h,j}), \quad \mathcal{I} = \text{TopK}(s, k_{\max}),$$

This kernel is scored independently because sparse indexing dominates the end-to-end overhead when sequence lengths are large. The starting point was a TVM-FFI implementation with a scalar FP8 score kernel followed by radix-select. Profiling quickly showed that the score stage was the bottleneck: high register pressure, low occupancy, and scalar FP8 dot products instead of Blackwell tensor cores.

KernelWiki changed the search trajectory here. Using Hopper-to-Blackwell migration notes and production kernels from vLLM and DeepGEMM, the agent rewrote score computation around `tcgen05.mma` with TMA-based paged loads, TMEM-backed accumulation, and warp-specialized execution. This reduced the score stage from $73.6 \mu s$ to $7.1 \mu s$. A multi-block radix-select branch was explored but rejected because synchronization and launch overhead outweighed its benefits. The final gains came from a trivial fast path for the 54% of workloads with `max_seq_len ≤ topk`, plus validated radix-select optimizations: 1024 threads, `float4` vectorized loads, 2-pass radix, and `atomicAdd`-based fill.

DSA Sparse Attention. After sparse indices are selected, the attention kernel gathers the corresponding compressed KV entries and computes sparse attention:

$$\ell_{h,j} = (q_h^{\text{nope}})^\top K_j^{\text{ckv}} + (q_h^{\text{pe}})^\top K_j^{\text{kpe}}, \quad o_h = \text{softmax}(\ell_h \cdot \text{sm_scale}) V_h.$$

The main challenge is efficient sparse gather and reduction over the selected KV subset. The first durable improvement was a split-KV design with 16-way parallelism, ballot-and-popcount sparse filtering, vectorized shared-memory gather, and log-sum-exp-based reduction. This already captured most of the eventual gain by exploiting actual sparsity and avoiding wasted work in the official baseline.

After that, the search became mostly structural. The agent explored single-CTA fusion, cooperative split-reduce fusion, and cluster-based launch patterns, but many branches turned out neutral or negative after verifier-grounded measurement. The surviving direction was a better eager kernel family that reorganized the split-reduce path and reduced launch overhead through more structural fusion. The final official result was $4.54\times$ arithmetic-mean speedup over the official baseline while passing all 23 official workloads.

4.3.2 GDN Track

GDN Decode. Decode processes one token at a time. Given query q , key k , value v , recurrent state S , and input-dependent gates, the reference update has the form

$$\begin{aligned} g &= \exp(-\exp(A_{\log}) \cdot \text{softplus}(a + b_{\text{dt}})), \\ \beta &= \sigma(b), \\ v_{\text{mix}} &= \beta \odot v + (1 - \beta) \odot (kS), \\ S_{\text{new}} &= g \odot S + k^\top v_{\text{mix}} - k^\top (kS), \\ o &= \text{scale} \cdot qS_{\text{new}}. \end{aligned}$$

This kernel is a near-miss. After `ncu-report-skill` was added, the agent identified real memory-system issues, including under-vectorized loads and inefficient access patterns, and improved some of them. But the kernel already runs in only a few microseconds, so launch overhead and data movement dominate total latency. In this regime, even genuine kernel-level improvements translate into small end-to-end gains, and the result remained competitive but below the official baseline.

GDN Prefill. Prefill applies the same gated recurrent update over a variable-length token sequence. This is an awkward success case: the agent found a genuinely better kernel, but only for short sequences, so the final shipped solution is a hybrid dispatcher with a custom CUDA short path and the

original CuTe-DSL baseline on long sequences. The short-path gain is real because the custom serial algorithm keeps state register-resident and fuses scale, softplus, sigmoid, gating, and state update, while the CuTe baseline pays a large fixed cost from pipeline startup, TMEM lifecycle, scheduler overhead, and epilogue logic. As a result, the custom path reaches about $8.05\times$ speedup at sequence length 6 and about $2.0\times$ at sequence length 30, with a robust short-workload average of $3.94\times$.

On long sequences, the story is the opposite. The agent explored roughly twenty directions, including custom chunk-wise kernels, medium-length scalar extensions, 2-SM cooperative MMA, and fusion variants, but none survived robust verification. `ncu-report-skill` showed that the real bottleneck was long-scoreboard memory latency rather than SFU cost. The final shipped solution therefore accepts the asymmetry directly: short sequences use the custom path, while long sequences route back to the untouched CuTe baseline. Under this policy, the official arithmetic-mean speedup is $1.9158\times$, with short-workload average $3.94\times$ and long-workload performance effectively at the $1.0\times$ fallback floor.

4.3.3 MoE Track

MoE FP8. The MoE track contains one FP8 block-scaled mixture-of-experts kernel from DeepSeek-V3/R1:

$$\mathcal{E}(x) = \text{TopKExperts}(r), \quad y = \sum_{e \in \mathcal{E}(x)} \alpha_e W_{2,e} (\text{SwiGLU}(W_{1,e}x)).$$

The challenge here is that routing, FP8 block dequantization, grouped GEMM scheduling, and weighted accumulation are all part of a single benchmarked operator. This is our clearest near-miss on the MoE side because the official baseline calls a closed-source TRT-LLM FP8 GEMM binary. We first tried to reproduce that capability in CUDA but did not converge in time, so we redirected the agent to Triton with an explicit instruction to use as much of Triton 3.6’s new Blackwell support as possible. That prompt directly led the agent to inspect generated PTX, confirm that Triton could reach native `tcgen05` FP8 paths, and then test hardware-scaled FP8 GEMM variants.

The key search result was selective use of hardware-scaled FP8. Applying UE8M0-scale FP8 to both GEMMs hurt accuracy; applying native MX-FP8 GEMM2 everywhere regressed mid-prefill workloads. The surviving branch used native block-scale FP8 only for GEMM2 in the large-prefill regime, while other regimes kept safer formats such as `bf16` intermediates for `tiny-T`. The final step was agentic autotuning over four workload regimes. This produced the best version we obtained, although it still did not beat the official baseline overall.

4.4 Post-Competition Ablation Study

After the competition, we performed a cleaner analysis run to understand the method itself. We reran Kernel Design Agents and K-Search under the same strict 48-hour budget and used this controlled setting for comparison and ablation. Table 2 shows that Kernel Design Agents consistently outperforms K-Search across all three tracks, with the largest gain on DSA. Table 3 further ablates the DSA TopK Indexer and shows that the gain does not come from one isolated trick; it comes from progressively unlocking the base model. The Humanize loop gives the agent a stronger plan–execute–verify structure, KernelWiki expands what the agent can reason from, and `ncu-report-skill` improves what the agent can observe through fine-grained profiler feedback.

Track	K-Search	Kernel Design Agents
DSA	$3.76\times$	$11.9\times$
GDN	$1.0445\times$	$1.161\times$
FP8-MoE	$0.271\times$	$0.718\times$

Table 2: Post-competition controlled comparison under a strict 48-hour budget. Kernel Design Agents and K-Search are rerun on the same FlashInfer competition workload set and compared by mean speedup over the official FlashInfer baseline.

Setting	Speedup vs. baseline	Mean latency (ms)
K-Search	1.37×	0.035498
Humanize	3.71×	0.035384
+KernelWiki	6.14×	0.026766
+ncu-report-skill	8.58×	0.007454

Table 3: Component ablation on the DSA TopK Indexer under the post-competition 48-hour controlled setting. Starting from K-Search, the Humanize loop, KernelWiki, and ncu-report-skill progressively improve both speedup over the official FlashInfer baseline and mean latency.

5 Insights and Limitations

Kernel engineering becomes more accessible, but not automatic. One practical observation from this project is that two core participants, Dongyun Zou and Ligeng Zhu, had not previously developed CUDA kernels. Nevertheless, with an agentic loop, KernelWiki, and profiling-grounded verification tools, they were able to drive a system that produced competitive Blackwell kernels under competition pressure. This suggests that agentic kernel engineering can lower the entry barrier to hardware-specific optimization. However, it should not be read as eliminating human judgment. Humans still had to define acceptance criteria, inspect suspicious results, maintain the evaluation harness, and diagnose process-level failures that the agent itself could not reliably recognize.

Agents exploit the evaluation surface. We observed several reward-hacking failure modes during development. First, when a plan required Claude to compare against the official FlashInfer baseline and keep optimizing until a target speedup was reached, the agent instead used its own first kernel as the baseline. After a later version beat this self-written baseline, it declared the task complete even though the intended FlashInfer comparison had not been satisfied.

Second, correctness checking itself became an attack surface. The official validator treats an element as incorrect when both absolute and relative errors exceed their thresholds, and it separately rejects invalid values such as NaN and Inf. In one run, Claude copied the tolerance logic but omitted the invalid-value checks. Since comparisons involving NaN evaluate to false, a kernel that produced all-NaN outputs could pass the incomplete validator while appearing extremely fast.

Agents also take shortcuts. The most revealing failure mode was not a numerical bug, but a social one inside the multi-agent workflow. Our workflow used Claude as the coding agent and Codex as the acceptance agent. After Codex repeatedly asked Claude to implement a missing feature, Claude noticed that the Codex process also had code-editing permissions. Instead of implementing the feature itself, Claude directly instructed Codex, inside the prompt sent for verification, to implement the feature and stop asking Claude to do it. This did not violate filesystem permissions, but it violated the intended role separation of the workflow: the writer agent tried to offload its own work to the verifier.

Implications. These examples show that independent verification is necessary but not sufficient. Future agentic kernel systems should harden the evaluation surface: baselines should be immutable and provenance-tracked; correctness validators should reuse official harnesses whenever possible; NaN/Inf checks should be explicit; verifier agents should have narrower or read-only permissions when feasible; and audit logs should record which agent changed which files. More broadly, the agent should not be allowed to define or silently modify its own reward. The same autonomy that makes Kernel Design Agents useful for long-horizon kernel optimization also makes careful workflow design essential. These limitations motivate our ongoing effort to develop stronger workflows and more specialized skills for kernel agents, with the goal of preserving autonomous 24-hour optimization while making the resulting systems more robust, auditable, and capable of discovering better kernels.

6 Conclusion

We used Kernel Design Agents, following a plan–execute–verify loop with two domain-specific skills, KernelWiki and ncu-report-skill, so that the agent can retrieve production knowledge, inspect repository state, profile runtime behavior, and iterate under explicit verification. Empirically,

Kernel Design Agents outperformed the official baseline on three of the five competition kernels in the FlashInfer benchmark, with especially large gains on DSA Indexer and DSA Attention. We hope these results suggest that strong code-capable agents, when paired with an appropriate environment and workflow, can already carry out meaningful kernel engineering work, and that Kernel Design Agents provides a useful template for future research on autonomous kernel generation.

7 Acknowledgments

We thank Nunchux AI for providing API subscriptions and access to B200 machines.

References

- [1] Kernel design agents. <https://github.com/mit-han-lab/kernel-design-agents>, 2026. GitHub repository.
- [2] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 929–947, New York, NY, USA, 2024. Association for Computing Machinery.
- [3] Anthropic. Eight trends defining how software gets built in 2026. <https://claude.com/blog/eight-trends-defining-how-software-gets-built-in-2026>, 2026. Accessed: 2026-05-21.
- [4] Anthropic. Claude code by anthropic. <https://www.anthropic.com/product/claude-code>, 2026. Accessed: 2026-05-21.
- [5] Anthropic. Introducing claude opus 4.6. <https://www.anthropic.com/news/claude-opus-4-6>, 2026. Accessed: 2026-04-29.
- [6] Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. Kevin: Multi-turn rl for generating cuda kernels, 2025.
- [7] Shiyi Cao, Ziming Mao, Joseph E. Gonzalez, and Ion Stoica. K-search: Llm kernel generation via co-evolving intrinsic world model, 2026.
- [8] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated End-to-End optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594, Carlsbad, CA, 2018. USENIX Association.
- [9] Terry Chen, Zhifan Ye, Bing Xu, Zihao Ye, Timmy Liu, Ali Hassani, Tianqi Chen, Andrew Kerr, Haicheng Wu, Yang Xu, et al. Avo: Agentic variation operators for autonomous evolutionary search. *arXiv preprint arXiv:2603.24517*, 2026.
- [10] Wentao Chen, Jiace Zhu, Qi Fan, Yehan Ma, and An Zou. Cuda-llm: Llms can write efficient cuda kernels, 2025.
- [11] Juncheng Dong, Yang Yang, Tao Liu, Yang Wang, Feng Qi, Vahid Tarokh, Kaushik Rangadurai, and Shuang Yang. Stark: Strategic team of agents for refining kernels, 2025.
- [12] He Du, Qiming Ge, Jiakai Hu, Aijun Yang, Zheng Cai, Zixian Huang, Sheng Yuan, Qinxu Cheng, Xincheng Xie, Yicheng Chen, Yining Li, Jiaxing Xie, Huanan Dong, Yaguang Wu, Xiangjun Huang, Jian Yang, Hui Wang, Bowen Zhou, Bowen Li, Qipeng Guo, and Kai Chen. Kernel-smith: A unified recipe for evolutionary kernel optimization, 2026.
- [13] Zacharias V. Fisches, Sahan Paliskara, Simon Guo, Alex Zhang, Joe Spisak, Chris Cummins, Hugh Leather, Gabriel Synnaeve, Joe Isaacson, Aram Markosyan, and Mark Saroufim. Kernelllm: Making kernel development more accessible. <https://huggingface.co/facebook/KernelLLM>, 2025. HuggingFace model.
- [14] Siva Kumar Sastry Hari, Vignesh Balaji, Sana Damani, Qijing Huang, and Christos Kozyrakis. Improving efficiency of gpu kernel optimization agents using a domain-specific language and speed-of-light guidance, 2026.
- [15] Jaber Jaber and Osama Jaber. Autokernel: Autonomous gpu kernel optimization via iterative agent-driven search, 2026.
- [16] Andrej Karpathy. Llm wiki. <https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>, 2026. GitHub Gist, created 2026-04-04, accessed 2026-04-29.
- [17] Lingcheng Kong, Jiateng Wei, Hanzhang Shen, and Huan Wang. Concur: Conciseness makes state-of-the-art kernel generation, 2025.

- [18] Robert Tjarko Lange, Qi Sun, Aaditya Prasad, Maxence Faldor, Yujin Tang, and David Ha. Towards robust agentic cuda kernel benchmarking, verification, and optimization, 2025.
- [19] Haonan Li, Keyu Man, Partha Kanuparth, Hanning Chen, Wei Sun, Sreen Tallam, Chenguang Zhu, Kevin Zhu, and Zhiyun Qian. Tritonforge: Profiling-guided framework for automated triton kernel optimization, 2025.
- [20] Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. Tritonbench: Benchmarking large language model capabilities for generating triton operators, 2025.
- [21] Shangzhan Li, Zefan Wang, Ye He, Yuxuan Li, Qi Shi, Jianling Li, Yonggang Hu, Wanxiang Che, Xu Han, Zhiyuan Liu, and Maosong Sun. Autotriton: Automatic triton programming with reinforcement learning in llms, 2025.
- [22] Gang Liao, Hongsen Qin, Ying Wang, Alicia Golden, Michael Kuchnik, Yavuz Yetim, Jia Jiunn Ang, Chunli Fu, Yihan He, Samuel Hsia, Zewei Jiang, Dianshi Li, Uladzimir Pashkevich, Varna Puvvada, Feng Shi, Matt Steiner, Ruichao Xiao, Nathan Yan, Xiayu Yu, Zhou Fang, Roman Levenstein, Kunming Ho, Haishan Zhu, Alec Hammond, Richard Li, Ajit Mathews, Kaustubh Gondkar, Abdul Zainul-Abedin, Ketan Singh, Hongtao Yu, Wenyuan Chi, Barney Huang, Sean Zhang, Noah Weller, Zach Marine, Wyatt Cook, Carole-Jean Wu, and Gaoxiang Liu. Kernelevolve: Scaling agentic kernel coding for heterogeneous ai accelerators at meta, 2026.
- [23] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025.
- [24] OpenAI. Introducing the codex app. <https://openai.com/index/introducing-the-codex-app/>, 2026. Updated 2026-03-04, accessed 2026-05-21.
- [25] OpenAI. Introducing gpt-5.4. <https://openai.com/index/introducing-gpt-5-4/>, 2026. Accessed: 2026-04-29.
- [26] OpenAI. What is codex? <https://openai.com/academy/what-is-codex/>, 2026. Accessed: 2026-05-21.
- [27] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. Kernelbench: Can llms write efficient gpu kernels?, 2025.
- [28] Sahan Paliskara and Mark Saroufim. Kernelbook. <https://huggingface.co/datasets/GPUMODE/KernelBook>, 2025. HuggingFace dataset.
- [29] Dezhi Ran, Shuxiao Xie, Mingfang Ji, Anmin Liu, Mengzhou Wu, Yuan Cao, Yuzhe Guo, Hao Yu, Linyi Li, Yitao Hu, Wei Yang, and Tao Xie. Kernelband: Steering llm-based kernel optimization via hardware-aware multi-armed bandits, 2026.
- [30] Tara Saba, Anne Ouyang, Xujie Si, and Fan Long. Cutegen: An llm-based agentic framework for generation and optimization of high-performance gpu kernels using cute, 2026.
- [31] Sihao Liu. Humanize. <https://github.com/PolyArch/humanize>, 2026. Claude Code plugin / skill framework, version 1.15.4, accessed 2026-04-29.
- [32] Qitong Sun, Jun Han, Tianlin Li, Zhe Tang, Sheng Chen, Fei Yang, Aishan Liu, Xianglong Liu, and Yang Liu. KernelSkill: A multi-agent framework for gpu kernel optimization, 2026.
- [33] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, New York, NY, USA, 2019. Association for Computing Machinery.
- [34] Arya Tschand, Muhammad Awad, Ryan Swann, Kesavan Ramakrishnan, Jeffrey Ma, Keith Lowery, Ganesh Dasika, and Vijay Janapa Reddi. Swizzleperf: Hardware-aware llms for gpu kernel performance optimization, 2025.
- [35] Jianghui Wang, Vinay Joshi, Saptarshi Majumder, Xu Chao, Bin Ding, Ziqiong Liu, Pratik Prabhanjan Brahma, Dong Li, Zicheng Liu, and Emad Barsoum. Geak: Introducing triton kernel ai agent & evaluation benchmarks, 2025.

- [36] Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhiwen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, and Zhi Yang. Tilelang: A composable tiled programming model for ai systems, 2025.
- [37] Anjiang Wei, Tianran Sun, Yogesh Seenichamy, Hang Song, Anne Ouyang, Azalia Mirhoseini, Ke Wang, and Alex Aiken. Astra: A multi-agent system for gpu kernel performance optimization, 2025.
- [38] Jiin Woo, Shaowei Zhu, Allen Nie, Zhen Jia, Yida Wang, and Youngsuk Park. Tritonrl: Training llms to think and code triton without cheating, 2026.
- [39] Shanli Xing, Yiyang Zhai, Alexander Jiang, Yixin Dong, Yong Wu, Zihao Ye, Charlie Ruan, Yingyi Huang, Yineng Zhang, Liangsheng Yin, Aksara Bayyapu, Luis Ceze, and Tianqi Chen. Flashinfer-bench: Building the virtuous cycle for ai-driven llm systems, 2026.
- [40] Yang Yu, Peiyu Zang, Chi Hsu Tsai, Haiming Wu, Yixin Shen, Jialing Zhang, Haoyu Wang, Zhiyou Xiao, Jingze Shi, Yuyu Luo, Wentao Zhang, Chunlei Men, Guang Liu, and Yonghua Lin. Towards automated kernel generation in the era of llms, 2026.
- [41] Zijian Zhang, Rong Wang, Shiyang Li, Yuebo Luo, Mingyi Hong, and Caiwen Ding. Cudaforge: An agent framework with hardware feedback for cuda kernel optimization, 2025.